



Welcome to this course where we will explore Phaser 4, an open-source HTML5 game framework designed for creating powerful and engaging 2D games for the web. Whether you are aiming to build your own games, develop interactive prototypes, or create portfolio-ready projects, Phaser 4 provides the necessary tools to bring your ideas to life.

What is Phaser 4?

Phaser 4 is a special framework for several key reasons:

- **Cross-Platform Compatibility:** You can build your game once and have it run anywhere. Phaser games are compatible with all modern web browsers.
- **Free and Open-Source:** Phaser is completely free to use and is supported by a thriving community of developers.
- **Ideal for 2D Games:** From platformers and arcade games to RPGs and virtual pets, Phaser 4 has the tools you need for 2D game development.
- **Modern JavaScript:** It is built on the latest JavaScript standards, making the development process efficient and enjoyable.
- **Feature-Rich:** We will explore a wide range of topics, including animations, timed events, player input, UI, Phaser scenes, and more, giving you everything required to build polished gameplay.

What You Will Be Building

Throughout this course, you will create your own virtual pet game from scratch using Phaser 4. In this game, you will manage a cute pixel pet by keeping it fed, entertained, and happy. You will add interactive UI buttons to provide it with food or toys, and you will even be able to drag the pet around the screen. However, if you neglect your pet for too long, its stats will drop, leading to a game over. This project is an excellent way to learn core Phaser 4 concepts.

By the end of this course, you will have a fully working game that you can expand upon with your own ideas or use as a launchpad for original game projects. You will learn not just how to build things, but why they work and how you can apply these concepts to your own games.

What You Will Learn

We will guide you through every step of the game development process. Key topics include:

- Working with sprite sheets and creating custom animations.
- Implementing drag-and-drop functionality for game objects.
- Using timed events to execute code at specific moments.
- Managing project complexity by working with multiple Phaser scenes.
- Building a loading screen with a custom progress bar for game assets.
- Bringing everything together with the core virtual pet game logic.

Prerequisites

Before we begin, here is what you will need to follow along with the course content:

- Basic to intermediate HTML, CSS, and JavaScript skills.
- Basic familiarity with Phaser 4 is helpful.
- A code editor, such as Visual Studio Code.
- A local web server to run your game files. We will cover how to set this up using the Live Server extension for VS Code early in the course.

If you already have a preferred IDE or web server setup, that will work as well.



About Zenva

Zenva is an online learning academy with over a million students, featuring a wide range of courses for both beginners and experienced developers looking to learn new skills. Our courses are versatile, allowing you to learn in the way that suits you best. You can choose to watch video tutorials, read lesson summaries, or follow along with the instructor using the included project files.

Are you ready to start building your own games? Let's dive in and begin our Phaser 4 journey.



Before beginning the development process, it is beneficial to understand the project we will be building in this course: a virtual pet game created with the Phaser framework. This article provides a high-level overview of the game concept, its core mechanics, and the planned structure for its scenes and user interface. Understanding this framework will provide context and clarity for the upcoming lessons.

Game Concept

Our game is inspired by classic virtual pet toys. The player's primary role is to take care of a digital creature by performing actions such as feeding it, playing with it, and generally keeping it happy. The pet will react to the player's actions. If it is well cared for, it will remain healthy and cheerful. Conversely, if the pet is neglected, its statistics will decline, eventually leading to a game over state.

Core Gameplay Mechanics

The following list outlines the core gameplay mechanics that will be implemented. Building these systems will teach essential Phaser skills, from handling user input and animations to implementing timed logic and UI feedback.

- **Draggable Pet:** The player will be able to move the pet around the screen using mouse or touch input.
- **Interactive Items:** Players can select items like food or toys and give them to the pet.
- **Stat Tracking:** The game will track the pet's core stats, such as health and fun, over time.
- **Timed Events:** The pet's stats will gradually decrease if the player does not interact with it, creating a sense of urgency.
- **Item Effects:** Each item will affect the pet's stats differently. Some will have positive effects, while others may have negative ones.
- **Game Over Condition:** If either the health or fun stat reaches zero, the game will end.

Game Structure with Phaser Scenes

To maintain a clean and organized codebase, the game will be divided into multiple Phaser Scenes. Using Phaser's Scene System makes each part of the game modular and easier to manage. Our structure will include the following scenes:

1. **Loading Scene:** This scene will be responsible for loading all game assets (images, sounds, etc.) and will display a progress bar to the user.
2. **Home Scene:** This will serve as the title screen, where the player must interact to begin the game.
3. **Game Scene:** This is the main scene where all core gameplay occurs.
4. **Game Over Scene:** This scene will be displayed when the game over condition is met (i.e., when the pet's health or fun reaches zero).

UI, Layout, and Responsiveness

A key goal is to support both desktop and mobile platforms. Therefore, the game must fill the screen and resize smoothly across different device sizes. Important UI elements, such as buttons and the pet itself, should always remain centered and visible regardless of the screen's dimensions.

To ensure the game functions correctly across various screen sizes, we will utilize Phaser's built-in Scale Manager. This powerful tool allows us to adjust the game canvas to different resolutions, maintain the game's aspect ratio, and help keep UI elements positioned correctly. Properly configuring the Scale Manager is a vital step in making the game feel polished and accessible on any device.



This concludes the overview of our virtual pet game. With a clear understanding of what we will be building and how it will be structured, we can now proceed with development.



In any development journey, the first and most crucial step is establishing a proper project environment. A solid foundation at this stage is essential for a smooth workflow and will help prevent complications later on. This article will guide you through setting up your development environment for creating games with Phaser.

Prerequisites

To begin, you will need to have a few tools and files ready. While we will be demonstrating with specific software, you are welcome to use alternatives if you have an existing setup you prefer.

- **A code editor:** We recommend using Visual Studio Code (VS Code), but any modern code editor is suitable.
- **A local web server:** We will use the Live Server extension for VS Code. This provides a simple and effective development server.
- **The project files:** Please download the starter project files and unzip them to an easily accessible location on your computer.

The Importance of a Local Web Server

As we create interactive games for the web using Phaser, we need a reliable way to view and test our progress. Simply opening the `index.html` file directly in a web browser can lead to limitations, especially when handling external resources and complex game logic. A local web server provides a more robust environment for development for several key reasons:

- **Serving Files Correctly:** A server properly serves your HTML, CSS, and JavaScript files, ensuring that all assets and dependencies are loaded as intended. This is critical for Phaser to function correctly.
- **Handling Relative Paths:** Web servers correctly interpret relative file paths within your project. This prevents issues where images, scripts, or other resources might fail to load when opening the HTML file directly from your file system.
- **Enabling Live Reloading:** Tools like Live Server can automatically refresh the browser whenever you save changes to your code. This instant feedback loop significantly speeds up the development process.

Installing the Live Server Extension

To streamline our development process, we will use the Live Server extension for VS Code. This tool automatically refreshes your web browser each time you save your code, making development much smoother and faster. To install it, please follow these steps:

1. Open VS Code and navigate to the Extensions view by clicking the icon that looks like four squares in the activity bar on the side.
2. In the search bar, type "Live Server".
3. Locate the extension by Ritwick Dey and click the "Install" button.
4. Once installed, a "Go Live" button should appear in the bottom-right corner of the VS Code status bar.

Exploring the Project Structure

Now that your tools are ready, let's examine the starter project files. Inside the main project folder, you will find several essential components for our game.

- **The Phaser Library:** This is the core of our game development framework. It is a collection of code that handles complex tasks for us, from rendering graphics to managing game logic.
- **index.html:** This file is the entry point for our game. It is a standard HTML document that



contains a head section for metadata and a body where our JavaScript files are linked and our game canvas will be rendered. Notice the two script tags that link to the Phaser library and our custom game code.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>? Virtual Pet</title>
  <style>
    html, body {
      background: black;
      padding: 0px;
      margin: 0px;
    }
  </style>
</head>
<body>
  <script src="src/lib/phaser.js"></script>
  <script src="src/main.js" type="module"></script>
</body>
</html>
```

- **assets/images/:** This folder contains all the image assets we will be using for our project.
- **main.js:** This is our main JavaScript file. Here, we create the Phaser game configuration and initialize a new Phaser game instance. This file links to our game scene.

```
import { GameScene } from './scenes/game-scene.js';
```

```
// our game's configuration
const config = {
  type: Phaser.AUTO,
  scale: {
    width: 360,
    height: 640,
    scene: [GameScene],
    title: 'Virtual Pet',
    pixelArt: false,
    backgroundColor: 'ffffff'
  }
};
```

```
// create the game, and pass it the configuration
const game = new Phaser.Game(config);
```

- **scenes/GameScene.js:** In this file, we define a Phaser Scene. For now, the scene's structure is set up, and we have preloaded the necessary images. The create method is currently empty, so nothing will be visible on the screen yet.

```
export class GameScene extends Phaser.Scene {
  constructor() {
    super({ key: 'GameScene' });
  }

  // some parameters for our scene
  init() {}
}
```



```
preload() {  
  // load assets  
  this.load.image('backyard', 'assets/images/backyard.png');  
  this.load.image('apple', 'assets/images/apple.png');  
  this.load.image('candy', 'assets/images/candy.png');  
  this.load.image('rotate', 'assets/images/rotate.png');  
  this.load.image('toy', 'assets/images/rubber_duck.png');  
}  
  
// executed once, after assets were loaded  
create() {}  
}
```

Running the Project

With our environment configured, let's view the project in a browser using Live Server.

1. In VS Code, with the project folder open, click the "Go Live" button in the bottom-right status bar.
2. VS Code will automatically open your default web browser and serve your project from a local server address, which typically looks like `http://127.0.0.1:5500`.

If everything is set up correctly, you should see a blank white rectangle in your browser. This is the canvas element where our game will be drawn. You can verify its existence by inspecting the web page's elements in your browser's developer tools; you will find a canvas element inside the code tag.

Recap

You now have a proper development environment set up and are ready to start building with Phaser. In this lesson, we have accomplished the following:

- Installed the Live Server extension in VS Code.
- Downloaded and reviewed the starter project files.
- Toured the project's structure and the purpose of each file.
- Used Live Server to run our blank game project in the browser.

In the next lesson, we will begin creating game objects for our game.



Welcome back. In this lesson, we will begin populating our game world by adding game objects to the scene. We will also explore how to load and utilize a sprite sheet in Phaser. This is a fundamental step that will later allow us to create animations and switch between different expressions for our virtual pet.

Adding Game Objects

To begin, we will modify our game scene. Please open the `game-scene.js` file located in the `src/scenes` folder. Since we have already loaded our image assets in the `preload` method, we can now reference them in the `create` method to display them in our game.

Let's start by adding the background. We will create a sprite game object for this purpose. In the `create` method, add the following code:

```
this.add.sprite(0, 0, 'backyard');
```

After saving, you may notice that only the top-left quadrant of the background image is visible. This occurs because, by default, Phaser places game objects based on their center point, or origin. Since we positioned our sprite at (0, 0), its center is at the top-left corner of the canvas, leaving most of the image off-screen.

To correct this, we must change the sprite's origin to its top-left corner. We can achieve this by chaining the `setOrigin` method.

```
this.add.sprite(0, 0, 'backyard').setOrigin(0, 0);
```

By calling `setOrigin(0, 0)`, we instruct Phaser to use the top-left corner of the image as its origin point for positioning. Now, when the sprite is placed at (0, 0), the entire background image will be correctly displayed on the canvas.

Creating the User Interface (UI)

Next, we will create the UI buttons that the player will use to interact with the pet. To maintain organized code, we will create a separate method for this task instead of placing all the logic within the `create` method.

First, call a new method named `createUi` from within your `create` method.

```
create() {  
  // game background  
  this.add.sprite(0, 0, 'backyard').setOrigin(0, 0);  
  
  this.createUi();  
}
```

Now, let's define the `createUi` method after the `create` method. Inside this new method, we will add the sprites for our four buttons: apple, candy, toy, and rotate.

```
createUi() {
```




```
// buttons
this.appleBtn = this.add.sprite(72, 570, 'apple');
this.candyBtn = this.add.sprite(144, 570, 'candy');
this.toyBtn = this.add.sprite(216, 570, 'toy');
this.rotateBtn = this.add.sprite(288, 570, 'rotate');
}
```

Here, we are creating four new sprite game objects and positioning them at the bottom of the screen with distinct x-coordinates to space them out. For these buttons, the default origin at the center of the image is suitable. We also store a reference to each button object as a property of our scene class (e.g., `this.appleBtn`). This will allow us to access and manipulate these buttons later in our code.

Loading and Using a Sprite Sheet

Our pet is not a single image but a sprite sheet, which is one large image composed of multiple smaller images, or frames. We need to instruct Phaser on how to load this asset and parse it into its individual frames.

This process should happen in the preload method to ensure the asset is ready before the create method is executed. We will use the `spritesheet` method from the loader plugin. This method requires a unique key, the path to the asset, and a configuration object that describes the layout of the frames.

In your `game-scene.js` file, within the preload method, add the following code to load the pet sprite sheet:

```
// load spritesheet
this.load.spritesheet('pet', 'assets/images/pet.png', {
  frameWidth: 97,
  frameHeight: 83,
  margin: 1,
  spacing: 1,
});
```

The configuration object specifies:

- `frameWidth`: The width of a single frame in pixels.
- `frameHeight`: The height of a single frame in pixels.
- `margin`: The space around the entire set of frames in the image.
- `spacing`: The space between individual frames.

With the sprite sheet loaded, we can now create the pet sprite in our `game-scene.js` file. Return to the create method and add the following code:

```
this.pet = this.add.sprite(100, 200, 'pet', 0);
```

When creating a sprite, you can provide an optional fourth argument, which is the frame index to display. Sprite sheet frames are indexed starting from 0. If you omit this argument, it defaults to 0,



displaying the first frame. By changing this value, you can display different frames from the sprite sheet. For example, using an index of 1 would show the second image of the pet.

Finally, we store a reference to the pet sprite in `this.pet`, just as we did with the UI buttons.

Challenge

Before moving on, here is a small challenge for you. Try changing the frame for the pet sprite to display a different expression. Besides setting the frame during creation, you can also use the `setFrame()` method on a sprite object at any time to update its displayed frame after it has been created.



Welcome! In the previous lesson, we presented a challenge to change the frame of your pet sprite. This article will walk you through a potential solution to that challenge.

To change the frame of a sprite, there are two primary methods. You can either specify the frame when the sprite is first created, or you can update it at any point after its creation using the `setFrame()` method.

Setting the Frame During Creation

When you create a sprite using `this.add.sprite()`, you can pass the desired frame index as the fourth argument. To match the challenge image, we need to use the fifth frame from our sprite sheet.

In Phaser, sprite sheet frames are indexed starting from zero. This means the first frame is at index 0, the second at index 1, and so on. Therefore, the fifth frame is at index 4.

To set this frame upon creation, you would modify the line where the pet sprite is created in your `create()` method:

```
this.pet = this.add.sprite(100, 200, 'pet', 4);
```

By changing the fourth argument from its previous value to 4, the pet sprite will now display the fifth frame from the sprite sheet as soon as the game loads.

Setting the Frame After Creation

Alternatively, you can change the sprite's frame after the game object has already been created. To do this, you first need a reference to your sprite object, which we have stored in `this.pet`. Then, you can call the `setFrame()` method on this object.

The `setFrame()` method takes the index of the frame you wish to display as its argument. To show the fifth frame (index 4), you would add the following line of code after the pet has been created:

```
this.pet.setFrame(4);
```

This approach is useful when you need to dynamically change the sprite's appearance in response to game events, such as a change in the pet's state.



In this lesson, we will discuss an important topic: correctly scaling your game. The Phaser Scale Manager helps control how your game content appears across different screen sizes, from large desktop monitors to smaller mobile devices. We will explore the scale configuration options and examine the different scale modes that Phaser provides.

Configuring the Scale Manager

The scale settings are placed inside your game configuration object when you initialize your Phaser game. In our current project, we are not yet using the Scale Manager. To enable it, we need to add a scale property to our configuration object. This property will hold an object containing our various scaling settings.

First, we will move the existing width and height properties from the top level of the configuration object into our new scale object. Then, we will add two new properties: `autoCenter` and `mode`.

The `autoCenter` property controls the positioning of your game's canvas element. We will set it to `Phaser.Scale.CENTER_BOTH`, which will horizontally and vertically center the canvas within the browser window.

The `mode` property defines how the game should be scaled. We will use `Phaser.Scale.FIT`, which scales the game to fit within the available space while maintaining its original aspect ratio.

Let's update our game configuration in `main.js`.

```
// our game's configuration
const config = {
  type: Phaser.AUTO,
  scale: {
    width: 360,
    height: 640,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    mode: Phaser.Scale.FIT,
  },
  scene: [GameScene],
  title: 'Virtual Pet',
  pixelArt: false,
  backgroundColor: 'ffffff'
};
```

Observing the Results

After saving these changes, you will immediately see that the game's canvas is now centered on the screen. The true power of the Scale Manager becomes evident when you simulate different device sizes. Using your browser's developer tools to emulate a mobile device, you will see that the game automatically scales to take up the full width and height available, adapting to the new dimensions.

If you were to temporarily remove the `autoCenter` and `mode` properties, you would notice that on a mobile device, the canvas appears very small relative to the screen size. On a desktop, it would retain its original dimensions without centering. By adding just two lines of code, Phaser's built-in Scale Manager handles the responsiveness for us across a variety of devices.

Reviewing the Scale Properties



Let's review the properties we added to the game configuration.

- **scale:** This property is an object that holds all the configuration settings for the Phaser Scale Manager.
- **width and height:** These properties set the default width and height of your game. This establishes the aspect ratio that the Scale Manager will try to maintain. While they can be set outside the scale object, it is considered best practice to include them within it when using the Scale Manager.
- **autoCenter:** This property determines where the canvas element is placed. By setting it to `Phaser.Scale.CENTER_BOTH`, we instruct Phaser to automatically center our game both horizontally and vertically.
- **mode:** This property defines the scaling strategy. `Phaser.Scale.FIT` ensures that the game canvas is scaled to fit inside the parent container while preserving the aspect ratio.



In this lesson, we will make our pet draggable, allowing the player to move it around the screen using either a mouse or a touchscreen. This involves making the pet interactive and then responding to drag events to update its position.

Making the Pet Interactive

To begin, we must make our pet sprite interactive. This tells the Phaser framework to register the game object for pointer events, such as clicks or touches. We can achieve this by calling the `setInteractive()` method on our pet game object right after its creation.

By default, simply making an object interactive does not make it draggable. We need to explicitly enable this behavior. We can do this by passing a configuration object to the `setInteractive()` method with the `draggable` property set to `true`.

```
this.pet = this.add.sprite(100, 200, 'pet', 0).setInteractive({
  draggable: true,
});
```

At this point, Phaser recognizes that the pet is draggable, but it does not yet know what to do when a drag occurs. If you were to run the game now, clicking and dragging the pet would have no visual effect.

Handling the Drag Event

To make the pet move with our pointer, we need to listen for Phaser's global drag event. We can access this through the scene's input plugin, which is responsible for all types of input, including mouse, keyboard, and touch events.

We will use the `on()` method of the input plugin to listen for a specific event and provide a callback function to execute when that event is fired. The event we are interested in is `Phaser.Input.Events.DRAG`.

Updating the Pet's Position

The callback function for the drag event receives several arguments. For our purposes, the most important are the game object being dragged, and the new x and y coordinates of the pointer, named `dragX` and `dragY` respectively.

Inside the callback, we can use these coordinates to update the position of our game object. We will call the `setPosition()` method on the dragged game object, passing in `dragX` and `dragY`.

Let's add the event listener in the `create()` method:

```
// follow pointer (mouse/finger) when dragging
this.input.on(Phaser.Input.Events.DRAG, (pointer, gameObject, dragX, dragY) => {
  // make sprite be located at the coordinates of the dragging
  gameObject.setPosition(dragX, dragY);
});
```

Now, when you save and run the game, you can click and hold on the pet and drag it anywhere on the screen. The pet's position will update in real-time to follow your pointer.



Lesson Recap

To summarize, we accomplished the following to make our pet draggable:

- We called the `setInteractive()` method on our pet's game object.
- We passed a configuration object `{ draggable: true }` to enable dragging.
- We set up a listener for the global `Phaser.Input.Events.DRAG` event using `this.input.on()`.
- In the event's callback function, we updated the game object's position using the provided `dragX` and `dragY` coordinates.



In this lesson, we will continue developing our game by implementing the logic that allows players to interact with the UI buttons. This will involve making the buttons responsive to clicks and executing specific actions for each button.

Making UI Elements Interactive

To enable player interaction with our UI elements, we must first make them interactive, just as we did with our pet game object. This is achieved by calling the `setInteractive()` method on each UI button's game object. We will do this within our `createUi` method.

```
createUi() {  
  // buttons  
  this.appleBtn = this.add.sprite(72, 570, 'apple').setInteractive();  
  this.candyBtn = this.add.sprite(144, 570, 'candy').setInteractive();  
  this.toyBtn = this.add.sprite(216, 570, 'toy').setInteractive();  
  this.rotateBtn = this.add.sprite(288, 570, 'rotate').setInteractive();  
}
```

By calling `setInteractive()`, we are signaling to Phaser that these game objects should listen for input events. Since these buttons will not be draggable, we do not need to pass any configuration arguments to the method.

Listening for Click Events

With input enabled, we now need to react when a button is clicked. For this, we will listen for the `pointerdown` event. This event is fired whenever a player clicks (or taps) on a game object that has been made interactive.

Unlike the global drag event we listened for on the input plugin, we will listen for the `pointerdown` event directly on each button's game object. This allows us to associate a specific callback function with a specific button, so we know exactly which UI element was clicked.

We can use the `on()` method on a game object to register an event listener. Let's start with the apple button as an example:

```
this.appleBtn.on(Phaser.Input.Events.POINTER_DOWN, () => {  
  console.log('apple button clicked');  
});
```

If you save your changes and click the apple icon in the game, you will see the "apple button clicked" message in the console. Clicking the other buttons will do nothing, as we have not yet registered any listeners for them.

Let's add a similar listener for the candy button:

```
this.candyBtn.on(Phaser.Input.Events.POINTER_DOWN, () => {  
  console.log('candy button clicked');  
});
```




Using Class Methods as Callbacks

While inline arrow functions are suitable for simple actions like logging a message, for more complex logic, it is better to use dedicated methods within our scene class. Let's refactor our code to use this approach for the remaining buttons.

We will create two new methods in our `GameScene` class: `pickItem` and `rotatePet`. We will then assign these methods as the callbacks for our toy and rotate buttons.

```
// In the createUi method
this.toyBtn.on(Phaser.Input.Events.POINTER_DOWN, this.pickItem);
this.rotateBtn.on(Phaser.Input.Events.POINTER_DOWN, this.rotatePet);
```

Now, let's define these new methods within the `GameScene` class. For now, they will simply log a message to the console.

```
// In the GameScene class
pickItem() {
  console.log('we are picking', this);
}

rotatePet() {
  console.log('we are rotating the pet!', this);
}
```

Understanding Callback Scope and Context

When you test the new buttons, you will notice that the value of `this` logged in the console is the sprite game object that was clicked, not our `GameScene` instance. This is the default behavior: when a game object's event triggers a callback, the scope (the context of `this`) within that callback is the game object itself.

This can be problematic if we need to access other properties or methods of our scene from within the callback. To solve this, the `on()` method accepts an optional third argument, which allows us to specify the context for the callback. We can pass `this` (referring to the scene instance) to ensure our method is executed with the correct scope.

Let's update the listener for the rotate button:

```
// The third argument binds the context of 'this' to our GameScene instance
this.rotateBtn.on(Phaser.Input.Events.POINTER_DOWN, this.rotatePet, this);
```

Now, when you click the rotate button, the `this` logged inside the `rotatePet` method will be the `GameScene`. This is crucial for allowing us to call other scene methods. For example, if we had a method called `test()` in our scene, we could call `this.test()` from within `rotatePet`. If we tried to do this from `pickItem` (which still has the game object as its context), it would result in an error because the sprite game object does not have a `test` method.

Finalizing the Click Handlers



Now we can finalize the implementation for all our buttons. First, let's update the apple and candy buttons to also use the `pickItem` method. Since we want to know which item was picked, we will rely on the default behavior where `this` refers to the clicked game object.

```
// In the createUi method, update the apple and candy button listeners
this.appleBtn.on(Phaser.Input.Events.POINTER_DOWN, this.pickItem);
this.candyBtn.on(Phaser.Input.Events.POINTER_DOWN, this.pickItem);
```

Next, let's modify the `pickItem` method to use the information from the game object. We can access the `texture` key to identify the item.

```
pickItem() {
    console.log('we are picking', this.texture.key);
}
```

Here, `this.texture` is a property Phaser adds to the game object, containing information about its texture. The `key` property holds the asset key we used when loading the image.

Finally, let's clean up the `rotatePet` method. Since we have already set its context correctly, we can simplify the log message.

```
rotatePet() {
    console.log('we are rotating the pet');
}
```

After these changes, clicking the apple, candy, or toy buttons will log a message identifying the picked item, and clicking the rotate button will log its specific action. Our UI buttons are now fully interactive, setting the stage for adding gameplay logic in our next lesson.



Welcome back. Now that our buttons are interactive, we will explore how to use them in our game. In a virtual pet game, the player's primary goal is to care for their pet. To visualize the pet's well-being, we can track statistics like hunger, happiness, or energy. These stats can be displayed as numerical counters, animated meters, or even through the pet's facial expressions and sprite changes.

The buttons we have added—the apple, candy, and toy—are the player's means of giving items to the pet. Each item will have a different effect. To manage these effects, we need a structured way to store data associated with each button.

The Pitfall of Manual Property Assignment

You might be tempted to add custom properties directly to a game object, like so:

```
// This is an example of what NOT to do
this.appleBtn.customStats = {
  health: 20
};
```

While this works in JavaScript, it is not an ideal approach. The main risk is accidentally overwriting built-in properties and methods on Phaser game objects. Since JavaScript is flexible, nothing prevents you from overwriting a critical property. For example, consider what happens if we assign our custom data to the `x` property:

```
// This will break the game object's positioning
this.appleBtn.x = {
  health: 20
};
console.log(this.appleBtn.x);
```

If you run this code, the apple button will disappear from the screen. This is because the `x` property is essential for Phaser to position game objects. By replacing its numerical value with an object, we have broken its positioning logic. To avoid such issues, Phaser provides a safer, built-in solution.

Using Phaser's Data Manager

Phaser includes a built-in Data Manager, a component that allows you to safely store, query, and retrieve key-value paired information on a game object. By default, game objects do not have a Data Manager instance; it is created automatically the first time you interact with it. The Data Manager is accessed through the `data` property on a game object.

Initially, this property is null:

```
console.log(this.appleBtn.data); // Outputs: null
```

To create the Data Manager instance and store data, we use the `setData()` method. This method takes a key and a value.

```
this.appleBtn.setData('test', 20);
```



When `setData()` is called, Phaser checks if a Data Manager exists. If not, it creates one and then stores the provided key-value pair. Now, if we log the data property, we will see a Data Manager instance.

The Data Manager can store various data types, including numbers, strings, and objects. You can also store multiple key-value pairs.

```
this.appleBtn.setData('test', 3);
this.appleBtn.setData('test2', { some: 'object' });
```

To retrieve a value, we use the `getData()` method, passing the key of the data we want to access.

```
console.log(this.appleBtn.getData('test')); // Outputs: 3
console.log(this.appleBtn.getData('test2')); // Outputs: { some: 'object' }
```

If you try to get data for a key that does not exist, Phaser will return undefined.

Reacting to Data Changes

A powerful feature of the Data Manager is its built-in event emitter. It emits events whenever its data changes, allowing you to react accordingly. For example, the `changedata` event is emitted anytime an existing property is updated.

We can listen for this event using the `on()` method:

```
this.appleBtn.on('changedata', (parent, key, value) => {
  console.log(key, value);
});

// Now, let's update a value to trigger the event
this.appleBtn.setData('test', 4); // This will log: "test", 4
```

This functionality is incredibly useful for tasks like updating the UI or an external data store whenever a stat changes, without needing to manually call update functions everywhere.

Implementing Stats for Our Buttons

Now, let's apply this to our game. We will store the health and fun effects for each of our item buttons. To ensure we use consistent keys across our code, we can define them as constants.

At the top of your `game-scene.js` file, before the class definition, add the following:

```
const CUSTOM_DATA_KEYS = Object.freeze({
  STATS: 'STATS'
});
```

Using `Object.freeze()` prevents this object from being modified after it's created, which helps prevent



accidental changes. Now, in the `createUi` method, we can use this key to store the stats for each button.

```
// In your createUi() method
this.appleBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 20, fun: 0 });

this.candyBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: -10, fun: 10 });

this.toyBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 0, fun: 15 });
```

Here, we've defined the effects of each item:

- The **apple** is healthy but not very fun.
- The **candy** is fun but unhealthy.
- The **toy** is very fun and has no effect on health.

Finally, let's update our `pickItem` method to log the stats of the clicked button. This confirms that our data is being stored and retrieved correctly.

```
pickItem(item) {
  console.log('we are picking ', item.texture.key);
  console.log(item.getData(CUSTOM_DATA_KEYS.STATS));
}
```

With these changes, when you click on any of the item buttons, you will see its associated stats object logged to the console.

Recap

In this lesson, we covered several important concepts for managing game data:

- We discussed why tracking stats is crucial for a virtual pet game.
- We learned about the risks of adding custom properties directly to game objects.
- We explored Phaser's Data Manager as a safe and robust alternative for storing custom data.
- We used the `setData()` and `getData()` methods to attach and retrieve stat values for our buttons.

Now that our buttons hold the data we need, we are ready to use this data to affect our pet. In the next lesson, we will continue working with our buttons and add states to keep track of which item is currently selected.



In this lesson, we will implement the logic for selecting items from the user interface (UI). We will also explore how the UI can react to different user interactions by using a simple state model. This model will help us manage when the player can and cannot interact with the UI elements.

A Simple UI State Model

For our game, we can conceptualize our UI as having three fundamental states: Ready, Selected, and Locked.

- **Ready:** This is the default state. The player can interact with the game, and all UI buttons are available to be clicked.
- **Selected:** This state is triggered when the player clicks on an item button. It indicates that an item has been chosen but not yet used.
- **Locked:** This state occurs when the pet is performing an action, such as eating food or playing with a toy. During this time, UI interactions are temporarily disabled to prevent conflicts. Once the action is complete, the UI will transition back to the Ready state.

We need to manage the transitions between these states as the player interacts with the game.

Tracking UI State

To begin, we will add two new properties to our GameScene class. One will track whether the UI is locked, and the other will keep a reference to the currently selected item. We will add these in the createUi method.

```
createUi() {
  // buttons
  this.appleBtn = this.add.sprite(72, 570, 'apple').setInteractive();
  // ...
  this.candyBtn = this.add.sprite(144, 570, 'candy').setInteractive();
  // ...
  this.toyBtn = this.add.sprite(216, 570, 'toy').setInteractive();
  // ...
  this.rotateBtn = this.add.sprite(288, 570, 'rotate').setInteractive();
  // ...

  this.isUiLocked = false;
  this.selectedItem = null;
}
```

Refactoring the pickItem Method

Currently, our pickItem method is bound to the scope of the clicked button. To access our new scene properties (isUiLocked and selectedItem), we need to change this. We will refactor the method to accept the clicked item as an argument and update our event listeners to pass this argument.

First, modify the pickItem method signature:

```
// pick item
pickItem(item) {
  // "this" scope is now referring to the GameScene instance
  console.log('we are picking ', item.texture.key);
  console.log(item.getData('STATS'));
```



```
}
```

Next, update the event listeners in `createUi` to use a callback function that calls `pickItem` with the correct button reference.

```
createUi() {
  // buttons
  this.appleBtn = this.add.sprite(72, 570, 'apple').setInteractive();
  this.appleBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.appleBtn); });
  this.appleBtn.setData('STATS', { health: 20, fun: 0 });

  this.candyBtn = this.add.sprite(144, 570, 'candy').setInteractive();
  this.candyBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.candyBtn); });
  this.candyBtn.setData('STATS', { health: -10, fun: 10 });

  this.toyBtn = this.add.sprite(216, 570, 'toy').setInteractive();
  this.toyBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.toyBtn); });
  this.toyBtn.setData('STATS', { health: 0, fun: 15 });

  // ...
}
```

Implementing Selection Logic

Now we can add the logic to the `pickItem` method. It will check if the UI is locked. If not, it will store the selected item and visually indicate the selection by reducing the button's alpha, making it semi-transparent.

```
// pick item
pickItem(item) {
  // "this" scope is now referring to the GameScene instance

  // the ui can't be locked in order to select an item
  if (this.isUiLocked) {
    return;
  }

  // keep track of the selected item
  this.selectedItem = item;

  // change transparency
  item.setAlpha(0.5);

  console.log('we are picking ', item.texture.key);
  console.log(item.getData('STATS'));
}
```



If you test this, you'll notice you can select multiple items. To fix this, we need a way to reset the UI before selecting a new item.

Refreshing the User Interface

We will create a new method called `refreshUi`. This method will be responsible for resetting the UI to its "Ready" state. It will clear the current selection, unlock the UI, and reset the appearance of all buttons.

To easily manage all UI buttons, let's first create an array property in `createUi` to hold them.

```
createUi() {
  // ... (button creation)

  this.uiButtons = [this.appleBtn, this.candyBtn, this.toyBtn, this.rotateBtn];

  this.isUiLocked = false;
  this.selectedItem = null;
}
```

Now, let's define the `refreshUi` method. As mentioned, this will reset everything and unlock the UI.

```
// unlock ui
refreshUi() {
  // nothing is being selected
  this.selectedItem = null;

  // set all buttons to alpha 1 (no transparency)
  this.uiButtons.forEach((button) => {
    button.setAlpha(1);
  });

  // scene must be unlocked
  this.isUiLocked = false;
}
```

Finally, we call `refreshUi` at the beginning of our `pickItem` method. This ensures that only one item can be selected at a time.

```
// pick item
pickItem(item) {
  // "this" scope is now referring to the GameScene instance

  // the ui can't be locked in order to select an item
  if (this.isUiLocked) {
    return;
  }

  // make sure the ui is ready
  this.refreshUi();
}
```




```
// keep track of the selected item
this.selectedItem = item;

// change transparency
item.setAlpha(0.5);

console.log('we are picking ', item.texture.key);
console.log(item.getData('STATS'));
}
```

Handling the Rotate Action

The rotate button triggers an immediate action rather than selecting an item to be placed. When clicked, it should lock the UI, perform the action, and then unlock the UI. We will simulate the action's duration using a delayed call.

Let's update the rotatePet method with this new logic.

```
// rotate pet
rotatePet() {
  // "this" scope is referring to the GameScene instance

  // the ui can't be locked in order to select an item
  if (this.isUiLocked) {
    return;
  }

  // make sure the ui is ready
  this.refreshUi();

  // lock the ui
  this.isUiLocked = true;

  // dim the rotate icon
  this.rotateBtn.setAlpha(0.5);

  // simulate a delay for the action
  this.time.delayedCall(2000, () => {
    this.refreshUi();
  });
}
```

With this implementation, clicking the rotate button will make it semi-transparent and lock the UI for two seconds. During this time, no other buttons can be clicked. After the delay, refreshUi is called, which resets the button's alpha and unlocks the UI.

We now have a functional UI state system that handles selecting items and blocking input during actions. In the next lesson, we will focus on placing the selected items into our scene.



In this lesson, we will implement the functionality to place items onto the background. After a player selects an item from the user interface, they will be able to click anywhere on the screen, and the selected item will appear at that location.

Making the Background Interactive

To begin, we need to enable our game's background to detect input events, such as clicks. We can achieve this by making the background sprite interactive. In the create method of your GameScene, locate where the background is created and call the `setInteractive()` method on it. It is also good practice to store the background game object in a variable for easier access.

```
// executed once, after assets were loaded
create() {
  // game background
  const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0).setInteractive();

  // ...
}
```

Listening for Clicks and Placing Items

Now that the background is interactive, we can listen for a "pointer down" event, which fires when the player clicks or taps on it. We will register an event listener that calls a new method, `placeItem`, whenever this event occurs. When registering the listener, we will also pass this as the final argument to ensure the `placeItem` method is called within the context of our scene.

```
// executed once, after assets were loaded
create() {
  // game background
  const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0).setInteractive();
  bg.on(Phaser.Input.Events.POINTER_DOWN, this.placeItem, this);

  // ...
}
```

Next, let's define the `placeItem` method at the bottom of our GameScene class. For now, we will add a simple console log to confirm it is being called correctly.

```
placeItem() {
  console.log('place item called');
}
```

If you run the game and click on the background, you should see the "place item called" message in your browser's console.

Implementing the Placement Logic

Before placing an item, we need to perform a couple of checks. First, we should ensure the UI is not locked. Second, we must verify that the player has actually selected an item. If either of these



conditions is not met, we will exit the method early using return.

The event callback for `POINTER_DOWN` provides several useful arguments. We are interested in `localX` and `localY`, which represent the coordinates of the click relative to the game object (in this case, our background). We will update our `placeItem` method to accept these arguments, along with the pointer object.

With the coordinates, we can create a new image game object. The image's texture should match the texture of the currently selected item.

```
placeItem(pointer, localX, localY) {
    // scene must be unlocked
    if (this.isUiLocked) {
        return;
    }
    // check that an item was selected
    if (this.selectedItem === null) {
        return;
    }

    // create a new item in the position the player clicked/tapped
    this.add.image(localX, localY, this.selectedItem.texture.key);
}
```

Tracking and Updating Pet Stats

Our game needs to track the pet's statistics, such as health and fun. We will initialize these stats in the `init` method of our scene. Let's create a stats object with initial values for health and fun.

```
// some parameters for our scene
init() {
    this.stats = {
        health: 100,
        fun: 100,
    };
    // ...
}
```

When an item is placed, we need to update these stats based on the data associated with that item. In the `placeItem` method, after creating the new item, we will retrieve the item's custom stats, loop through them, and apply the changes to our pet's main stats. We will also add a check to ensure we only modify stats that already exist on our pet's stats object.

Finally, after placing an item and updating the stats, we will call `this.refreshUi()` to reset the interface and allow the player to select another item.

```
placeItem(pointer, localX, localY) {
    // scene must be unlocked
    if (this.isUiLocked) {
        return;
    }
}
```



```
// check that an item was selected
if (this.selectedItem === null) {
  return;
}

// create a new item in the position the player clicked/tapped
this.add.image(localX, localY, this.selectedItem.texture.key);

// update the pet stats
const customStats = this.selectedItem.getData('STATS');
for (const [key, value] of Object.entries(customStats)) {
  if (this.stats[key] !== undefined) {
    this.stats[key] += value;
  }
}
console.log(this.stats);

// unlock ui
this.refreshUi();
}
```

Now, when you select an item like the apple and place it, you will see the pet's health increase in the console. If you place the candy, you will see the health decrease and fun increase, demonstrating that our stat system is working.

Recap and Challenge

In this lesson, we have accomplished the following:

- Made the background game object interactive.
- Listened for pointer events to detect when the player clicks the background.
- Implemented logic to place a new item at the clicked location.
- Created a system to track the pet's health and fun stats.
- Updated the pet's stats based on the data of the placed item.
- Reset the UI after an item was placed.

For your challenge, consider the current implementation of the `placeItem` method. It creates a new game object every time an item is placed. Since our game will only have one item on the screen at a time for the pet to interact with, refactor the `placeItem` method to reuse the same game object for placing items rather than creating a new one each time. This will be a valuable optimization for later lessons.



In our previous lesson, we issued a challenge to refactor the `placeItem` method. The goal was to reuse the same item game object each time we place an item, rather than creating a new instance. This refactoring is an important optimization step, making our game more efficient by reusing game objects instead of creating and destroying them repeatedly.

Storing the Placed Item

To begin, we need a way to track the item's game object after it has been created. The best approach is to add a new property to our `GameScene` class. This property will hold the reference to the created item.

Let's modify the `placeItem` method. We will replace the logic that creates a new item on every call. Instead, we will first check if `this.placedItem` already exists. If it does, we will reuse it; otherwise, we will create it for the first time.

The check involves seeing if `this.placedItem` is not equal to `undefined`. If the property is no longer `undefined`, it means we have already created our image game object and can simply update its position. If it is still `undefined`, we proceed with creating the image and assigning it to `this.placedItem`.

```
placeItem(pointer, localX, localY) {
    // ... other code in the method

    // create a new item in the position the player clicked/tapped
    if (this.placedItem !== undefined) {
        this.placedItem.setPosition(localX, localY);
    } else {
        this.placedItem = this.add.image(localX, localY, this.selectedItem.texture.key);
    }

    // ... other code in the method
}
```

With this change, if you test the game, you will notice that clicking the apple button places the apple. Clicking it again moves the same apple object to the new location. However, if you then select the candy button, the apple object moves, but its appearance does not change to a candy. We need to update the object's texture as well.

Updating the Item's Texture

To fix the texture issue, we need to update the texture of our game object whenever we reuse it. After setting the position of the existing `placedItem`, we can call the `setTexture` method on it. This method takes the key of the new texture we want to display, which we can get from our `selectedItem`.

Let's add the call to `setTexture` inside the `if` block in our `placeItem` method.

```
placeItem(pointer, localX, localY) {
    // ... other code in the method

    // create a new item in the position the player clicked/tapped
    if (this.placedItem !== undefined) {
        this.placedItem.setPosition(localX, localY);
        this.placedItem.setTexture(this.selectedItem.texture.key);
    }
}
```



```
        this.placedItem.setTexture(this.selectedItem.texture.key);
    } else {
        this.placedItem = this.add.image(localX, localY, this.selectedItem.texture.key);
    }

    // ... other code in the method
}
```

Now, if you run the game, everything should work as expected. You can place an item, move it, and when you select a different item, the game object will move to the new position and its texture will update correctly. Our method is now successfully refactored and optimized.



In this lesson, we will bring our pet to life by animating it with a rotation tween when the rotate button is clicked. We will also enhance the button's functionality to boost our pet's fun stat, and we will refactor our code to make our stat-updating logic more reusable.

Adding Stats to the Rotate Button

Currently, our rotate button does not affect the pet's stats. To change this, we first need to assign custom stat data to it, just as we did for the apple, candy, and toy buttons. In the createUi method, we can use the setData method on our rotate button to store the values we want to apply.

Let's modify the rotate button's creation in createUi to include a fun value of 10.

```
this.rotateBtn = this.add.sprite(288, 570, 'rotate').setInteractive();
// the third argument allows us to pass what scope we want to be bound to the method
provide
this.rotateBtn.on(Phaser.Input.Events.POINTER_DOWN, this.rotatePet, this);
this.rotateBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 0, fun: 10 });
```

Refactoring Stat Updates

Now that the rotate button has stat data, we need to apply it. We also apply stats when an item is placed on the screen. To avoid writing the same logic in multiple places, we can create a single, reusable method called updateStats. This method will take a stats object and apply its values to our pet's main stats.

Let's create the updateStats method. It will iterate through the properties of the passed-in customStats object and update the corresponding properties in this.stats.

```
// stat updater
updateStats(gameObject) {
  // update the pet stats
  const customStats = gameObject.getData(CUSTOM_DATA_KEYS.STATS);
  for (const [key, value] of Object.entries(customStats)) {
    // console.log(key, value);
    if (this.stats[key] !== undefined) {
      this.stats[key] += value;
    }
  }
}
```

With this new method, we can simplify the logic inside our placeItem and rotatePet methods. We will first get the data from the relevant button and then pass it to our new updateStats method.

```
placeItem(pointer, localX, localY) {
  // existing code

  this.updateStats(this.selectedItem)

  this.refreshUi();
}

rotatePet() {
  // "this" scope is referring to the GameScene instance
```



```
// the ui can't be locked in order to select an item
if (this.isUiLocked) {
    return;
}

// make sure the ui is ready
this.refreshUi();

// lock the ui
this.isUiLocked = true;

// dim the rotate icon
this.rotateBtn.setAlpha(0.5);

this.time.delayedCall(2000, () => {
    this.refreshUi();
    this.updateStats(this.rotateBtn);
});

console.log('we are rotating the pet!');
}
```

Introducing Phaser Tweens

To animate our pet, we will replace the simple timed delay in our `rotatePet` method with a Phaser Tween. The Tween Manager in Phaser allows us to create smooth, time-based animations by changing an object's properties over a set duration. For example, we can animate an object's position, scale, alpha (transparency), or rotation.

We will use a tween to animate the angle property of our pet sprite, making it spin. To create a tween, we use `this.tweens.add()` and pass it a configuration object.

Implementing the Rotation Animation

Let's modify the `rotatePet` method to use a tween. The tween will target our pet, animate its angle to 720 degrees over 600 milliseconds, and then call a function to update the stats and UI once it's complete.

Here are the key properties for our tween configuration:

- **targets:** The game object(s) to animate. In our case, `this.pet`.
- **angle:** The property we want to animate. We will set it to 720 to make the pet spin twice.
- **duration:** The length of the animation in milliseconds. We'll use 600.
- **onComplete:** A callback function that executes when the tween finishes. This is where we will update the stats and refresh the UI.
- **callbackScope:** The context (the value of `this`) for the `onComplete` callback. We'll set it to `this` to ensure we can access our scene's methods.

Here is the updated `rotatePet` method:

```
// rotate pet
rotatePet() {
    // "this" scope is referring to the GameScene instance
```




```
// the ui can't be locked in order to select an item
if (this.isUiLocked) {
    return;
}

// make sure the ui is ready
this.refreshUi();

// lock the ui
this.isUiLocked = true;

// dim the rotate icon
this.rotateBtn.setAlpha(0.5);

// rotation tween
this.tweens.add({
    targets: this.pet,
    angle: 720,
    duration: 600,
    onComplete: () => {
        // set UI to ready
        this.refreshUi();
        this.updateStats(this.rotateBtn);
    },
    callbackScope: this,
});

console.log('we are rotating the pet!');
}
```

If you run the game now and click the rotate button, you will see the pet sprite spin smoothly. After the animation completes, the pet's fun stat will increase, and the UI will be unlocked.

Tweens are very flexible. For example, you could animate multiple properties at once. If we wanted the pet to fade out while spinning, we could add an alpha property to the tween configuration:

```
this.tweens.add({
    targets: this.pet,
    angle: 720,
    alpha: 0, // Add this line
    duration: 600,
    onComplete: () => {
        this.pet.setAlpha(1); // Reset alpha
        // ... update stats and refresh UI
    },
    callbackScope: this,
});
```

For our game, we will stick with just the rotation. The main takeaway is that the Tween Manager is a powerful tool for creating a wide variety of animations in your game. With our rotation animation



complete, we are ready for the next step.



In this lesson, we will complete our pet's interaction behavior. When an item is placed in the game, we will make our pet walk towards it. Upon arrival, the pet will play a custom animation to "consume" the item, its stats will be updated, and then it will return to its default state. Let's bring this sequence to life.

Enhancing the `placeItem` Method

Currently, when an item is placed, our UI is immediately refreshed. We want to change this behavior. Instead, we will lock the UI to prevent player input while the pet is moving towards and interacting with the item.

In your `game-scene.js` file, navigate to the `placeItem` method. After the initial checks for a locked UI and a selected item, we will lock the UI by setting `isUiLocked` to `true`.

```
placeItem(pointer, localX, localY) {
    // scene must be unlocked
    if (this.isUiLocked) {
        return;
    }
    // check that an item was selected
    if (this.selectedItem === null) {
        return;
    }

    this.isUiLocked = true;

    // ...
}
```

Animating Pet Movement with Tweens

To make the pet move smoothly to the placed item, we will use a tween. This is similar to how we animated the pet's rotation. We can copy the tween creation logic from the `rotatePet` method and adapt it for our needs.

In the `placeItem` method, after the code that places the item in the scene, add the tween. We will modify it to animate the pet's `x` and `y` properties to match the coordinates of the `placedItem`. The duration will be set to 500 milliseconds. The logic to update stats and refresh the UI will now happen in the `onComplete` callback of the tween.

```
this.tweens.add({
    targets: this.pet,
    x: this.placedItem.x,
    y: this.placedItem.y,
    duration: 500,
    onComplete: () => {
        this.updateStats(this.rotateBtn);
        // set UI to ready
        this.refreshUi();
    },
    callbackScope: this,
});
```



We should update the stats before refreshing the UI. We will make this correction in the rotate item tween so the behavior between our two functions is consistent.

```
rotatePet() {
  // ... existing code ...

  // rotation tween
  this.tweens.add({
    targets: this.pet,
    angle: 720,
    duration: 600,
    onComplete: () => {
      // increase fun
      this.updateStats(this.rotateBtn);
      // set UI to ready
      this.refreshUi();
    },
    callbackScope: this,
  });

  console.log('we are rotating the pet!');
}
```

Consuming and Hiding the Item

Once the pet reaches the item, the item should disappear as if it has been consumed. We can achieve this by hiding the game object. Inside the tween's onComplete callback, we will use the setVisible(false) method on our placedItem.

```
onComplete: () => {
  // hide our item game object
  this.placedItem.setVisible(false);

  // ... rest of code ...
},
```

Fixing Item Visibility

If you test the game now, you will notice a bug: after the first item is placed and hidden, placing another item does not make it appear. This is because we are reusing the same placedItem game object, which is still set to be invisible.

To fix this, we need to explicitly set its visibility to true whenever we place it. In the placeItem method, find the block of code that handles repositioning an existing placedItem and add a call to setVisible(true).

```
// create a new item in the position the player clicked/tapped
if (this.placedItem !== undefined) {
  this.placedItem.setPosition(localX, localY);
  this.placedItem.setTexture(this.selectedItem.texture.key);
  this.placedItem.setVisible(true);
} else {
```



```
this.placedItem = this.add.image(localX, localY, this.selectedItem.texture.key);
}
```

Creating Custom Animations

Phaser allows us to create animations by sequencing frames from a spritesheet. We will create an animation that makes it look like our pet is eating. To do this, we must first define the animation so Phaser knows about it.

In the create method of GameScene, before the UI is created, let's add our animation definition. We use the animation manager, this.anims, and its create method.

The create method takes a configuration object with several properties:

- **key:** A unique string to identify the animation. We'll name it 'funnyfaces'.
- **frames:** An array of frames that make up the animation. We can use `this.anims.generateFrameNames()` to easily generate these from our 'pet' spritesheet, specifying the frame indices we want to use (1, 2, and 3).
- **frameRate:** The speed of the animation in frames per second. We'll use 7.
- **yoyo:** If set to true, the animation will play forwards and then in reverse, creating a looping effect.
- **repeat:** The number of times the animation should repeat. A value of 0 means it will play once (the initial play plus zero repeats).

```
// executed once, after assets were loaded
create() {
  // ...

  // create animation
  this.anims.create({
    key: 'funnyfaces',
    frames: this.anims.generateFrameNames('pet', { frames: [1, 2, 3] }),
    frameRate: 7,
    yoyo: true,
    repeat: 0, // to repeat forever: -1
  });

  // create ui
  this.createUi();
  // ...
}
```

Playing the Animation

Now that the animation is defined, we can play it on our pet sprite. We will trigger this animation when the pet reaches the item. In the onComplete callback of our movement tween, after hiding the item, call the play() method on the pet sprite, passing the key of the animation we want to play.

```
onComplete: () => {
  // hide our item game object
  this.placedItem.setVisible(false);
  // play spritesheet animation
}
```



```
    this.pet.play('funnyfaces');  
    // ... rest of code...  
},
```

With this final piece of code, your pet will now walk to the placed item, play an eating animation, and have its stats updated. In the next lesson, we will wait for this animation to complete before unlocking the UI for the player.



In this lesson, we will finalize our pet's behavior. We will ensure that the user interface (UI) remains locked until our pet's animations are complete, and we will address some visual and logical refinements to improve the game's flow.

Waiting for Animation Completion

A key aspect of our game's interaction is to prevent the player from performing new actions while the pet is reacting to a previous one. To achieve this, we need to wait until the pet's "eating" animation finishes before unlocking the UI. Phaser provides a robust event system that we can leverage for this purpose.

When an animation plays on a sprite, Phaser emits several events. The one we are interested in is the `ANIMATION_COMPLETE` event, which fires once the animation has finished its full cycle. We can listen for this event on our pet game object.

To set up the listener, we use the `on()` method of our pet sprite. The first argument is the event name, and the second is the callback function to execute when the event occurs.

```
this.pet.on(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {  
    // set UI to ready  
    this.refreshUi();  
}, this);
```

However, we should not create a new event listener every time we play the animation. A more efficient approach is to set up this listener just once when the pet is created. Let's move this logic to our scene's `create` method, right after we initialize the pet sprite animation.

```
// in create()  
// event listener for when spritesheet animation ends  
this.pet.on(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {  
    // set UI to ready  
    this.refreshUi();  
}, this);
```

Resetting the Pet's Frame

After the eating animation completes, you may notice that the pet's face remains on the last frame of the animation. We want the pet to return to its neutral expression (the first frame, index 0) after the action is done. We can achieve this by calling the `setFrame()` method within our `ANIMATION_COMPLETE` event listener, just before we refresh the UI.

```
this.pet.on(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {  
    // set pet back to neutral face  
    this.pet.setFrame(0);  
    // set UI to ready  
    this.refreshUi();  
}, this);
```

Managing Render Order with Depth



Another visual issue is that when we place an item, our pet moves to it but appears underneath the item sprite. By default, Phaser renders game objects in the order they are added to the scene. To control this rendering order, we can use the depth property.

All game objects have a default depth of 0. An object with a higher depth value will be rendered on top of an object with a lower depth value. By setting our pet's depth to a value greater than 0, we can ensure it always appears on top of the items. Let's set its depth to 1 in the create method.

```
// in create()
this.pet = this.add.sprite(100, 200, 'pet', 0).setInteractive({
  draggable: true,
});
this.pet.setDepth(1);
```

Fixing a Logic Bug

During development, a small bug was introduced from copying and pasting code. In our `placeItem` method, the call to `updateStats` was incorrectly using the stats from the rotate button instead of the currently selected item. We need to correct this to ensure the proper health and fun values are applied.

Inside the `onComplete` callback of the tween in the `placeItem` method, find the line that updates the stats and ensure it references `this.selectedItem`.

```
// inside the tween in the placeItem method
onComplete: () => {
  // hide our item game object
  this.placedItem.setVisible(false);
  // play spritesheet animation
  this.pet.play('funnyfaces');
  // increase stats
  this.updateStats(this.selectedItem);
},
```

With these changes, our pet's core behavior is complete. We have learned how to:

- Use tweens to move game objects across the screen.
- Create and play custom animations from a spritesheet.
- Listen for built-in Phaser events like `ANIMATION_COMPLETE` to trigger subsequent logic.
- Manage the rendering order of game objects using depth.

In the next lesson, we will focus on creating a Heads-Up Display (HUD) to visually represent the pet's health and fun stats to the player.



In this lesson, we will construct a heads-up display, or HUD, to show the player the current health and fun stats of their pet. For our HUD, we will use Phaser's built-in text game objects to display these statistics.

Creating the HUD

To begin, we will add a new method to our `GameScene` class specifically for creating the HUD. Let's add a method named `createHud` at the bottom of the class.

```
createHud() {  
  
}
```

This new method needs to be called from our main `create` method. We will call it immediately after we create our UI elements.

```
// in create()  
this.createUi();  
this.createHud();
```

Adding Text Elements

Inside the `createHud` method, we can create text game objects using the game object factory: `this.add.text()`. This method requires a few arguments:

1. The x-coordinate for the text's position.
2. The y-coordinate for the text's position.
3. The string of text to display.
4. A configuration object for styling the text.

Let's add our first text object for the "Health" stat. For now, we will use an empty configuration object.

```
// in createHud()  
this.add.text(20, 20, 'Health:', {});
```

If you run the game now, you will see the text appear in the top-left corner. By default, Phaser renders text in white with a default font and size. We can customize this appearance using the configuration object.

- `fill`: Sets the text color.
- `fontSize`: Sets the size of the font.
- `fontFamily`: Sets the font family.

Let's apply some styles to make our text black, larger, and use the Arial font.

```
// in createHud()  
this.add.text(20, 20, 'Health:', {  
  fill: '#000000',
```



```
fontSize: '24px',  
fontFamily: 'Arial',  
});
```

An important detail to note is that, unlike other game objects that have their origin in the center, text game objects have their origin in the top-left corner. This affects how you position them in your scene.

Now, let's create another text object for the "Fun" stat. We can copy the previous code and adjust the text and x-position.

```
// in createHud()  
this.add.text(170, 20, 'Fun:', {  
  fill: '#000000',  
  fontSize: '24px',  
  fontFamily: 'Arial',  
});
```

Displaying Dynamic Stat Values

Currently, our HUD only displays static labels. To show the actual stat values, we need a way to update the text dynamically. We will create a new method, `refreshHud`, for this purpose. We use a separate method because we'll need to call it any time the pet's stats change.

First, we must store references to our text game objects so we can access them later. We'll assign them to properties of our scene, such as `this.healthText` and `this.funText`.

```
createHud() {  
  this.healthText = this.add.text(20, 20, 'Health: ', {  
    fill: '#000000',  
    fontSize: '24px',  
    fontFamily: 'Arial',  
  });  
  
  this.funText = this.add.text(170, 20, 'Fun: ', {  
    fill: '#000000',  
    fontSize: '24px',  
    fontFamily: 'Arial',  
  });  
}
```

Now, let's define the `refreshHud` method. Inside this method, we will use the `setText()` method of our text objects to update their content. We can use template literals to easily combine our labels with the stat values from `this.stats`.

```
refreshHud() {  
  this.healthText.setText(`Health: ${this.stats.health}`);  
  this.funText.setText(`Fun: ${this.stats.fun}`);  
}
```



Updating the HUD

To see the initial values when the game starts, we need to call our new method from the create method, right after we create the HUD.

```
// in create()
this.createUi();
this.createHud();
this.refreshHud();
```

With this change, the HUD will now display the initial health and fun values. The final step is to ensure the HUD updates whenever the stats change. We can achieve this by calling refreshHud from within our existing refreshUi method, which is executed after a pet interaction is complete.

```
// in refreshUi()
// ...
this.isUiLocked = false;

// update hud
this.refreshHud();
```

Now, when you play the game and interact with your pet, you will see the health and fun stats update in real-time on the screen.

Recap

In this lesson, we successfully added text elements to our game to display the pet's fun and health stats. We created a reusable refreshHud method and ensured it is called every time our pet's stats change, keeping the display synchronized with the game state.



In this lesson, we will explore Phaser's timed events. We will use these events to make our pet's stats, specifically health and fun, decay over time. If either of these stats reaches zero, our game will end. Let's begin.

Phaser's Time Plugin

To have our stats decay over time, we need a way to instruct Phaser to execute a piece of code after a specific duration. Phaser's time plugin is the perfect tool for this. It allows us to create a time event, which schedules code to run at a given time. We can configure this event to run once after a delay or to repeat at regular intervals.

To create our first timed event, navigate to the create method in your GameScene. After the UI elements are created, we can add our timer.

```
// in the create() method
this.time.addEvent({
  delay: 1000,
  callback: function() {
    console.log('this ran');
  },
  callbackScope: this
});
```

Here, we use `this.time.addEvent()` and pass it a configuration object with the following properties:

- **delay:** The time in milliseconds that Phaser will wait before triggering the event. Here, 1000 milliseconds equals one second.
- **callback:** The function that will be executed when the event is triggered.
- **callbackScope:** The context (the value of `this`) to be applied to the callback function. We set it to `this` to refer to our scene instance.

If you run the game now, you will see the message "this ran" in the console one second after the game starts.

Repeating Timed Events

Besides scheduling a one-time event, we can also make it repeat. By adding the `repeat` property, we can control how many times the event re-occurs. For example, a value of 1 will cause the event to run once initially, wait for the delay, and then run one more time.

If we set `repeat` to -1, the event will repeat indefinitely every second for the entire duration of the game.

```
this.time.addEvent({
  delay: 1000,
  callback: function() {
    console.log('this ran');
  },
  callbackScope: this,
  repeat: -1 // Repeats forever
});
```



Implementing Stat Decay

Now that our timer is working, let's use it to decay our pet's stats. First, we need to define how much each stat should decrease. In the init method, let's add a new decayRates object.

```
init() {
  this.stats = {
    health: 100,
    fun: 100,
  };
  // decay parameters
  this.decayRates = {
    health: -5,
    fun: -2,
  };
}
```

Next, we need to update our updateStats method. Currently, it is designed to work with the stats from our UI buttons. To make it more versatile, we will refactor it to accept a stats object as an argument, rather than retrieving it from a game object internally.

Change the method signature of updateStats to the following:

```
// stat updater
updateStats(customStats) {
  // update the pet stats
  for (const [key, value] of Object.entries(customStats)) {
    // console.log(key, value);
    if (this.stats[key] !== undefined) {
      this.stats[key] += value;
    }
  }
  console.log(this.stats);
}
```

Now, you must update the places where updateStats was called, which in this case is where we rotate the pet and place an item.

```
// rotate pet
rotatePet() {
  // ... rest of code
  onComplete: () => {
    // increase fun
    const customStats = this.rotateBtn.getData(CUSTOM_DATA_KEYS.STATS);
    this.updateStats(customStats);
    // set UI to ready
    this.refreshUi();
  },
  callbackScope: this,
});
console.log('we are rotating the pet!');
```



```
}

placeItem(pointer, localX, localY) {
  // ... rest of code
  onComplete: () => {
    // hide our item game object
    this.placedItem.setVisible(false);
    // play spritesheet animation
    this.pet.play('funnyfaces');
    // increase stats
    const customStats = this.selectedItem.getData(CUSTOM_DATA_KEYS.STATS);
    this.updateStats(customStats);
  },
  callbackScope: this,
});
}
```

With our method refactored, we can now modify our timed event to decay the stats and refresh the display.

```
// in the create() method
this.time.addEvent({
  delay: 1000,
  repeat: -1,
  callback: () => {
    this.updateStats(this.decayRates);
    this.refreshHud();
  },
  callbackScope: this,
});
```

Now, every second, the pet's health will decrease by 5 and its fun will decrease by 2. Interacting with items will still increase the stats as before.

Handling Game Over

When a stat reaches zero, the game should be over. We will add this logic to our `updateStats` method. Inside the method, we will add a check to ensure a stat never drops below zero. If it does, we will reset it to zero and flag that the game is over.

Let's modify the `updateStats` method:

```
updateStats(customStats) {
  let isGameOver = false;

  for (const [key, value] of Object.entries(customStats)) {
    if (this.stats[key] !== undefined) {
      this.stats[key] += value;

      if (this.stats[key] <= 0) {
        this.stats[key] = 0;
      }
    }
  }
}
```



```
        isGameOver = true;
    }
}

if (isGameOver) {
    this.handleGameOver();
}
}
```

Finally, let's create the new `handleGameOver` method. For now, it will simply log a message to the console.

```
handleGameOver() {
    console.log('game over');
}
```

If you let the game run, you will see the “game over” message appear in the console as soon as either health or fun reaches zero.

Recap

In this lesson, we successfully implemented a core gameplay mechanic. We learned how to use Phaser's timer events to schedule repeating code. We refactored our `updateStats` method to be more reusable, allowing it to handle both stat increases from items and stat decay over time. Finally, we laid the groundwork for our game-over condition. In the next lesson, we will build upon this by creating a home screen and connecting our scenes to complete the game loop.



Welcome to the lesson. Here, we will create a home screen for our game. This screen will serve as the starting point for players, featuring a background, styled text, and an interactive “tap to start” element. Upon interaction, the game will transition to the main game scene. We will also prepare our game logic to eventually transition back from the game scene to the home screen upon a “game over” condition.

Creating a New Scene

To keep our project organized and scalable, it is good practice to isolate different parts of our game into separate scenes. Instead of adding all the home screen logic into our existing `GameScene`, we will create a new, dedicated scene for it. In Phaser, you can create as many scenes as you need.

Let’s begin by creating a new file in the `src/scenes/` folder named `home-scene.js`. Inside this new file, we will define a new class called `HomeScene` that extends the base `Phaser.Scene` class.

Every scene needs a unique key to identify it. We provide this key in the `constructor` by calling `super()` with a configuration object.

```
export class HomeScene extends Phaser.Scene {
  constructor() {
    super({ key: 'HomeScene' });
  }

  create() {
    console.log('HomeScene');
  }
}
```

Integrating the New Scene

Now that we have our new scene class, we must connect it to our game. This is done in the `main.js` file, where our Phaser game configuration is defined.

First, import the `HomeScene` at the top of `main.js`.

```
import { HomeScene } from '../scenes/home-scene.js';
import { GameScene } from '../scenes/game-scene.js';
```

Next, to make our `HomeScene` the first scene that loads when the game starts, we need to add it to the scene array in the game configuration object, placing it before `GameScene`.

```
const config = {
  // ... other config properties
  scene: [HomeScene, GameScene],
  // ... other config properties
};
```




If you save your changes and refresh the game, you should see a blank white screen and the “HomeScene” message in your browser’s developer console, confirming that our new scene is loading correctly.

Building the Home Screen UI

With the scene set up, let’s start building the user interface. We will add a title text to the center of the screen. First, remove the `console.log` from the `create` method. Then, we can get the game’s dimensions from the Scale Manager.

```
create() {  
    const { width, height } = this.scale;  
  
    const text = this.add.text(width / 2, height / 2, 'Virtual Pet', {  
        fill: 'ffffff',  
        fontSize: '40px',  
        fontFamily: 'Arial'  
    });  
}
```

If you save and run this, you’ll notice the text is not perfectly centered. This is because the origin point of a text game object is at its top-left corner by default. To center it, we need to change its origin to its center point (0.5, 0.5).

```
const text = this.add.text(width / 2, height / 2, 'virtual pet', {  
    fill: 'ffffff',  
    fontSize: '40px',  
    fontFamily: 'Arial'  
}).setOrigin(0.5);
```

Adding a Visual Background

The white text is not visible against the default white background. To make it stand out, let’s add a semi-transparent black rectangle behind it. We can use the `rectangle` game object for this.

We will position the rectangle at the same coordinates as our text and size it to be slightly larger than the text, creating a padding effect. The color will be black (0x000000).

```
this.add.rectangle(text.x, text.y, text.width + 20, text.height + 20, 0x000000);
```

When you add this code, you’ll see the rectangle, but the text will disappear. This is because game objects are drawn in the order they are created. To fix this, we can set the “depth” of the text object, forcing it to be drawn on top of the rectangle.

```
const text = this.add.text(/* ... */).setOrigin(0.5).setDepth(1);
```



Finally, to make the rectangle's background transparent, we can provide an alpha value as the last argument to the `rectangle` method. An alpha of 0.7 will provide a nice translucent effect.

```
this.add.rectangle(text.x, text.y, text.width + 20, text.height + 20, 0x000000, 0.7);
```

Loading and Displaying the Background Image

Our home screen should use the same background image as our game scene. However, assets loaded in one scene are not automatically available in another. We must load the background image within our `HomeScene`. To do this, we'll add a `preload` method to `HomeScene.js` and load the `'backyard'` image.

```
preload() {  
    this.load.image('backyard', 'assets/images/backyard.png');  
}
```

Now, in the `create` method, we can add the background sprite to the scene.

```
create() {  
    const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0);  
  
    const { width, height } = this.scale;  
    // ... rest of the create method  
}
```

Transitioning to the Game Scene

The final piece of functionality for our home screen is to transition to the `GameScene` when the player clicks or taps. We will add an event listener for a pointer down event. When this event is triggered, we will use the Scene Manager's `start` method to stop the current scene and start the `GameScene`.

Let's make the background interactive and add the event listener to it.

```
const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0).setInteractive();  
  
// ...  
  
bg.on(Phaser.Input.Events.POINTER_DOWN, () => {  
    this.scene.start('GameScene');  
}, this);
```



Currently, this event will fire every time the player clicks. Since we only want the transition to happen once, we should use a one-time event listener. Phaser provides the `once` method for this purpose. It works just like `on`, but automatically removes the listener after it has been triggered once.

```
bg.once(Phaser.Input.Events.POINTER_DOWN, () => {  
  this.scene.start('GameScene');  
}, this);
```

Final Touches

Let's make a couple of small improvements. First, we'll update the title text to include an emoji for a bit more personality.

```
const text = this.add.text(width / 2, height / 2, '? VIRTUAL PET', {  
  // ... style properties  
}).setOrigin(0.5).setDepth(1);
```

Second, since we are now loading the 'backyard' image in our `HomeScene` (which runs before `GameScene`), we no longer need to load it in `GameScene.js`. You can go into `GameScene.js` and remove the following line from its `preload` method to avoid loading the same asset twice:

```
this.load.image('backyard', 'assets/images/backyard.png');
```

Summary

In this lesson, we have successfully built a polished and interactive home screen. Let's recap what we covered:

- We learned how to add multiple Phaser scenes to a project by creating a new scene class and updating the game configuration.
- We used the rectangle game object to create a simple background element for our text, adding visual contrast.
- We saw how to manage draw order using the `setDepth` method.
- We implemented scene transitions using `this.scene.start()`.
- We learned how to use the `once` method to register a one-time event listener.

In the next lesson, we will finalize the game over logic and make the `GameScene` transition back to this `HomeScene`.



In this lesson, we will complete the game over logic by blocking input, displaying a sad pet, and transitioning out of the game. We will also refactor our asset loading and animation creation into a separate scene for better code organization.

Finalizing the Game Over Logic

Currently, our `handleGameOver` method in `GameScene` only logs a message to the console. We need to expand this to provide a proper game over sequence. The sequence will be as follows:

1. Lock the UI to prevent further player interaction.
2. Update the pet's sprite to show a sad expression.
3. Stop the stats from decaying further.
4. Wait for a few seconds.
5. Transition back to the home screen.

Let's begin by modifying the `handleGameOver` method. First, we will lock the UI and change the pet's frame. The sad face for our pet is the last frame in its spritesheet, which is at index 4.

```
handleGameOver() {  
  // block ui  
  this.isUiLocked = true;  
  
  // change frame of the pet  
  this.pet.setFrame(4);  
}
```

If you test the game now, you might notice that even after the game is over, the pet's stats continue to decrease. This is because the timed event we created for stat decay is still running. To fix this, we must store a reference to this timer when we create it and then destroy it when the game is over.

In the `create` method of `GameScene`, find where the timer is created and store it in a class property.

```
// in create()  
this.timerEventForStateDecay = this.time.addEvent({  
  delay: 1000,  
  repeat: -1, // it will repeat forever  
  callback: () => {  
    // update stats & refresh hud  
    this.updateStats(this.decayRates);  
    this.refreshHud();  
  },  
  callbackScope: this,  
});
```

Now, we can use this reference to destroy the timer in our `handleGameOver` method.

```
handleGameOver() {  
  // block ui  
  this.isUiLocked = true;  
  
  // cleanup timer event for stat decay
```



```
this.timerEventForStateDecay.destroy();

// change frame of the pet
this.pet.setFrame(4);
}
```

Another small issue is that the game over logic triggers only when a stat drops *below* zero, not when it reaches exactly zero. We can fix this with a small change in the `updateStats` method. Modify the condition to check if the stat is less than or equal to zero.

```
// in updateStats()
if (this.stats[key] <= 0) {
    this.stats[key] = 0;
    isGameOver = true;
}
```

Finally, let's add a delay before transitioning back to the `HomeScene`. We can achieve this by creating a new, non-repeating timed event inside `handleGameOver`.

```
handleGameOver() {
    // block ui
    this.isUiLocked = true;

    // cleanup timer event for stat decay
    this.timerEventForStateDecay.destroy();

    // change frame of the pet
    this.pet.setFrame(4);

    // keep the game on for a some time, the move on
    this.time.addEvent({
        delay: 2000,
        repeat: 0,
        callback: () => {
            this.scene.start('HomeScene');
        },
        callbackScope: this,
    });
}
```

With these changes, our game over sequence is now complete.

Refactoring Asset Loading

When you restart the game by transitioning from the `GameScene` back to the `HomeScene` and then starting a new game, you may notice a warning in the console about an animation key already existing. This happens because the `preload` and `create` methods of a scene are executed every time the scene starts. This means we are attempting to load assets and create animations that have already been processed.



A best practice is to move all one-time setup logic, like loading assets and creating animations, into a dedicated scene that runs only once at the beginning of the game. Let's create a LoadingScene for this purpose.

Creating the Loading Scene

First, create a new file named loading-scene.js inside the src/scenes/ folder. You can use the structure of HomeScene as a template.

```
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  preload() {
    // Asset loading will go here
  }

  create() {
    // Animation creation and scene transition will go here
  }
}
```

Next, we need to add this new scene to our game's configuration in main.js. It's important to place it before the other scenes so it runs first.

```
// in main.js
import { LoadingScene } from '../scenes/loading-scene.js';
// ... other imports

const config = {
  // ...
  scene: [LoadingScene, HomeScene, GameScene],
  // ...
};
```

Moving Assets and Animations

Now, let's move the relevant code into our new LoadingScene. Cut all the code from the preload method in GameScene and paste it into the preload method of LoadingScene. You can then delete the now-empty preload method from GameScene.

```
preload() {
  // load assets
  this.load.image('backyard', 'assets/images/backyard.png');
  this.load.image('apple', 'assets/images/apple.png');
  this.load.image('candy', 'assets/images/candy.png');
  this.load.image('rotate', 'assets/images/rotate.png');
  this.load.image('toy', 'assets/images/rubber_duck.png');

  // load spritesheet
```



```
this.load.spritesheet('pet', 'assets/images/pet.png', {
  frameWidth: 97,
  frameHeight: 83,
  margin: 1,
  spacing: 1,
});
}
```

Next, cut the animation creation logic (`this.anims.create(...)`) from the `create` method in `GameScene` and paste it into the `create` method of `LoadingScene`.

```
create() {
  // create animation
  this.anims.create({
    key: 'funnyfaces',
    frames: this.anims.generateFrameNames('pet', { frames: [1, 2, 3] }),
    frameRate: 7,
    yoyo: true,
    repeat: 0, // to repeat forever: -1
  });
}
```

Since all assets are now loaded in `LoadingScene`, you can also remove the `preload` method from `HomeScene` if it contains any asset loading.

After loading assets and creating animations, the `LoadingScene` should transition to the `HomeScene`. Add the following line to the end of the `create` method in `LoadingScene`.

```
// in LoadingScene's create() method
create() {
  // create animation
  this.anims.create({
    key: 'funnyfaces',
    frames: this.anims.generateFrameNames('pet', { frames: [1, 2, 3] }),
    frameRate: 7,
    yoyo: true,
    repeat: 0,
  });

  this.scene.start('HomeScene');
}
```

Final Testing and Bug Fixing

After refactoring, it's crucial to test the entire game flow. A common issue that can arise relates to object references that persist between scene transitions.

If you play the game, let it end, and then start a new game, you might encounter an error when trying to place an item. This is because the `this.placedItem` property in `GameScene` still holds a



reference to the item's game object from the previous game. Although Phaser destroys the game object when the scene shuts down, the JavaScript reference remains, causing issues when the scene restarts.

To fix this, we must reset the property to ensure it's clean for the new game. Add the following line to the create method in GameScene.

```
// in GameScene's create() method  
this.placedItem = undefined;
```

This ensures that every time the GameScene starts, placedItem is in a predictable state, preventing errors from the previous session.

With that, our core game logic is complete and well-organized. In the next lesson, we will enhance our LoadingScene by adding a visual progress bar.



In this lesson, we will build a proper loading screen for our game, complete with a logo. This is a small but important step that helps deliver a smoother player experience by providing visual feedback while game assets are being loaded.

The Problem with Long Loading Times

Currently, when our game starts, Phaser immediately begins loading the assets defined in our LoadingScene. Since we only have a few assets, this process is very fast. However, as a game grows in size and complexity, the number and size of assets increase, leading to longer load times. During this period, the player sees a blank white screen, which might make them think the game is broken or has frozen.

To simulate this effect, we can temporarily add a loop to our LoadingScene.js file to load a large number of assets. Add the following code inside the preload method:

```
// In scenes/loading-scene.js

// ... inside the preload() method
this.load.spritesheet('pet', 'assets/images/pet.png', {
  frameWidth: 97,
  frameHeight: 83,
  margin: 1,
  spacing: 1,
});

// Test code to simulate long loading
for (let i = 0; i < 2000; i += 1) {
  this.load.image(`toy${i}`, 'assets/images/rubber_duck.png');
}
```

If you save and refresh the game, you will notice a significant delay with a blank screen before the game appears. To solve this, we will create a loading screen that informs the player that the game is loading.

A Two-Step Loading Process

To display an image on our loading screen, that image must first be loaded. We will implement a two-step loading process using two different scenes:

- **Boot Scene:** A new scene whose only responsibility is to load the minimal assets required for the loading screen itself (like a company or game logo). This scene loads very quickly.
- **Loading Scene:** This scene will display the assets loaded by the Boot Scene (our logo) and then proceed to load all the other game assets.

This structure ensures that the player sees something on the screen almost immediately, improving the user experience.

Creating the Boot Scene

First, let's create the new scene file. In the src/scenes folder, create a new file named boot-scene.js. We will start by copying the contents of loading-scene.js into our new file and then modifying it.

Update the code in boot-scene.js to define the BootScene class. Its preload method will only load our



logo image, and its create method will immediately start the LoadingScene.

```
// In src/scenes/boot-scene.js
export class BootScene extends Phaser.Scene {
  constructor() {
    super({ key: 'BootScene' });
  }

  preload() {
    // load assets for the loading screen
    this.load.image('logo', 'assets/images/rubber_duck.png');
  }

  create() {
    this.scene.start('LoadingScene');
  }
}
```

Updating the Game Configuration

Next, we need to tell Phaser about our new BootScene and ensure it runs first. Open src/main.js, import the new scene, and add it to the beginning of the scene array in the game configuration.

```
// In src/main.js
import { GameScene } from './scenes/game-scene.js';
import { HomeScene } from './scenes/home-scene.js';
import { LoadingScene } from './scenes/loading-scene.js';
import { BootScene } from './scenes/boot-scene.js';

// our game's configuration
const config = {
  // ... other config properties
  scene: [BootScene, LoadingScene, HomeScene, GameScene],
  // ... other config properties
};

// create the game, and pass it the configuration
const game = new Phaser.Game(config);
```

Displaying the Logo

Now that the BootScene has loaded our logo, we can display it in the LoadingScene while the rest of the game assets are loaded in the background. Open src/scenes/loading-scene.js and add code to its preload method to display the logo image. We will also remove the test loop we added earlier.

```
// In src/scenes/loading-scene.js
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  preload() {
```



```
// game width and height
const { width, height } = this.scale;

// show logo
this.add.image(width / 2, 250, 'logo');

// load assets
this.load.image('backyard', 'assets/images/backyard.png');
this.load.image('apple', 'assets/images/apple.png');
this.load.image('candy', 'assets/images/candy.png');
this.load.image('rotate', 'assets/images/rotate.png');
this.load.image('toy', 'assets/images/rubber_duck.png');

// load spritesheet
this.load.spritesheet('pet', 'assets/images/pet.png', {
  frameWidth: 97,
  frameHeight: 83,
  margin: 1,
  spacing: 1,
});
}

// ... create() method remains the same
}
```

With these changes, when you run the game, the BootScene will load the logo instantly, and then the LoadingScene will display it while loading everything else. The blank white screen is now replaced by our game's logo, providing a much more professional feel. In our next lesson, we will enhance this screen by adding a progress bar.



Our loading screen requires a progress bar to provide visual feedback to the player as the game assets are loaded. This bar will be composed of two rectangle GameObjects. The first will be a static, light gray background representing the empty bar. The second will be a light green rectangle that will increase in width, from left to right, to show the loading progress.

Creating the Progress Bar Rectangles

We will add the progress bar elements within the preload method of our LoadingScene. First, let's define the dimensions for our bar to avoid using hardcoded numbers throughout our code. We will also add the background rectangle, which will be centered on the screen.

```
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  preload() {
    // game width and height
    const { width, height } = this.scale;
    // show logo
    this.add.image(width / 2, 250, 'logo');

    const barW = 150;
    const barH = 30;
    // progress bar background
    this.add.rectangle(width / 2, height / 2, barW, barH, 0xf5f5f5, 1);

    // ...
  }

  // ...
}
```

Next, we will add the second rectangle, which will serve as the fill color. This rectangle will be light green. We will store a reference to it in a constant named `progressBar` so we can modify it later. Its initial width will be set to 0, as no assets have been loaded yet.

To make the bar fill from left to right, we must adjust its origin point. By default, transformations like scaling occur from the center of a GameObject. We will use the `setOrigin()` method to change the origin to the left-center edge by passing the values 0 for the x-axis and 0.5 for the y-axis. Because we changed the origin, we also need to adjust its horizontal position to ensure it aligns correctly with the background rectangle.

```
// progress bar
const progressBar = this.add.rectangle(width / 2 - barW/2, height / 2, 0, barH, 0xad98d, 1).setOrigin(0, 0.5);
```

Listening for Loading Progress

Phaser's Loader plugin emits several built-in events during the asset loading process. To update our progress bar, we will listen for the PROGRESS event. This event is fired as each file in the loading



queue is processed.

We can set up a listener using the `this.load.on()` method. The listener's callback function receives a value parameter, which is a number between 0 and 1 representing the total loading progress.

```
// listen to the "progress" event
this.load.on(Phaser.Loader.Events.PROGRESS, (value) => {
  // we will update the progress bar here
});
```

Updating the Progress Bar Width

Inside the callback function for our progress listener, we can use the value to dynamically set the width of our progress bar rectangle. We will call the `setSize()` method on our progress bar object. The new width will be the total progress (value) multiplied by the bar's maximum width (barW). The height will remain constant.

Let's see the complete preload method with the progress bar logic implemented.

```
preload() {
  // game width and height
  const { width, height } = this.scale;
  // show logo
  this.add.image(width / 2, 250, 'logo');

  const barW = 150;
  const barH = 30;
  // progress bar background
  this.add.rectangle(width / 2, height / 2, barW, barH, 0xf5f5f5, 1);
  // progress bar
  const progressBar = this.add.rectangle(width / 2 - barW/2, height / 2, 0, barH, 0xad98d, 1).setOrigin(0, 0.5);

  // load assets
  this.load.image('backyard', 'assets/images/backyard.png');
  this.load.image('apple', 'assets/images/apple.png');
  this.load.image('candy', 'assets/images/candy.png');
  this.load.image('rotate', 'assets/images/rotate.png');
  this.load.image('toy', 'assets/images/rubber_duck.png');

  // load spritesheet
  this.load.spritesheet('pet', 'assets/images/pet.png', {
    frameWidth: 97,
    frameHeight: 83,
    margin: 1,
    spacing: 1,
  });

  // listen to the "progress" event
  this.load.on(Phaser.Loader.Events.PROGRESS, (value) => {
    progressBar.setSize(value * barW, barH);
  });
}
```



With this implementation, the green progress bar will now fill up from left to right as your game assets are loaded, providing a polished and professional loading screen for your players.



Congratulations on completing this course and your journey into Phaser 4 game development. Over the past lessons, we have explored Phaser 4, an open-source, modern JavaScript framework that has enabled us to build a complete, interactive browser game from the ground up. Along the way, you have acquired essential game development skills and created a project you can be truly proud of.

Why Phaser 4?

Phaser 4 was the perfect tool for this project for several key reasons. It provides a powerful and flexible environment for creating 2D games that run directly in the browser.

- **Cross-Platform Compatibility:** It runs directly in any modern web browser, making your game accessible to a wide audience without any special installations.
- **Open-Source and Free:** There are no licensing fees or limitations, just a great community and complete creative freedom.
- **Designed for 2D Games:** It is ideal for creating simulation, arcade, and casual games.
- **Modern JavaScript:** It is built with modern JavaScript, which provides features that make development clean and productive.
- **Rich Feature Set:** From animation and input to scenes and tweens, Phaser provides all the necessary tools for game creation.

Your Virtual Pet Game

Together, we built a charming virtual pet game where players care for a pet by feeding it, entertaining it, and keeping it healthy and happy. You successfully implemented core mechanics that brought this experience to life, including interactive UI buttons, drag-and-drop pet movement, animated sprite reactions, timed stat decay, and game-over logic. By the end of the course, you created a fully playable experience and hopefully gained ideas for ways you could expand upon it.

A Recap of Your Achievements

Let's quickly review the key concepts and skills we accomplished together throughout this course.

1. We saw how to import sprite sheets and use them to create custom animations in our game.
2. We learned about handling player input, specifically how to make game objects draggable and interactive.
3. We explored timed events, which allowed us to implement stat decay to simulate the ongoing needs of our pet.
4. We learned how to build a more complex game by utilizing multiple Phaser scenes to manage different game states.
5. We created a custom progress bar to display while loading all the necessary game assets.
6. Finally, we implemented the core logic required to make our virtual pet game functional and engaging.

Continue Your Learning Journey

You now have the foundational skills to bring your own game ideas to life using Phaser. Here at Zenva, we feature a wide range of courses for people who are just starting out or for those who want to expand their skills. Our courses are versatile, allowing you to learn by viewing video tutorials, reading lesson summaries, or following along with the included project files.

Thank you again for joining this course. We hope you enjoyed building your virtual pet game and feel inspired to continue creating with Phaser. Do not stop here; experiment with new mechanics, explore different game concepts, and continue to grow your skills. Until next time, happy coding!



In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

main.js

Found in folder: **/src**

This code imports four scenes: GameScene, HomeScene, LoadingScene, and BootScene. It then defines a configuration object for the game, specifying the game's type, scale, scenes, title, pixel art setting, and background color. Finally, it creates a new Phaser.Game instance, passing in the configuration object.

```
import { GameScene } from './scenes/game-scene.js';
import { HomeScene } from './scenes/home-scene.js';
import { LoadingScene } from './scenes/loading-scene.js';
import { BootScene } from './scenes/boot-scene.js';

// our game's configuration
const config = {
  type: Phaser.AUTO,
  scale: {
    width: 360,
    height: 640,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    mode: Phaser.Scale.FIT,
  },
  scene: [BootScene, LoadingScene, HomeScene, GameScene],
  title: 'Virtual Pet',
  pixelArt: false,
  backgroundColor: 'ffffff'
};

// create the game, and pass it the configuration
const game = new Phaser.Game(config);
```

boot-scene.js

Found in folder: **/src/scenes**

This code defines a BootScene class that extends Phaser.Scene. The preload() method loads an image asset, and the create() method starts the 'LoadingScene'.

```
export class BootScene extends Phaser.Scene {
  constructor() {
    super({ key: 'BootScene' });
  }

  preload() {
    // load assets
```




```
    this.load.image('logo', 'assets/images/rubber_duck.png');
  }

  create() {
    this.scene.start('LoadingScene');
  }
}
```

game-scene.js

Found in folder: **/src/scenes**

This code defines a `GameScene` class that extends `Phaser.Scene`. The class initializes the game scene with parameters for health and fun decay, creates the game background, pet sprite, and UI elements, and sets up event listeners for user interactions. The code also includes methods for updating stats, refreshing the UI and HUD, and handling game over conditions.

```
const CUSTOM_DATA_KEYS = Object.freeze({
  STATS: 'STATS'
});

export class GameScene extends Phaser.Scene {
  constructor() {
    super({ key: 'GameScene' });
  }

  // some parameters for our scene
  init() {
    this.stats = {
      health: 100,
      fun: 100,
    };
    // decay parameters
    this.decayRates = {
      health: -5,
      fun: -2,
    };
  }

  // executed once, after assets were loaded
  create() {
    // game background
    const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0).setInteractive();
    bg.on(Phaser.Input.Events.POINTER_DOWN, this.placeItem, this);

    this.pet = this.add.sprite(100, 200, 'pet', 0).setInteractive({
      draggable: true,
    });
    // you can also make a game object draggable by using the setDraggable method
    // this.input.setDraggable(this.pet);
    this.pet.setDepth(1);

    // event listener for when spritesheet animation ends
```



```
this.pet.on(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
    // set pet back to neutral face
    this.pet.setFrame(0);
    // set UI to ready
    this.refreshUi();
}, this);

// create ui
this.createUi();
this.createHud();
this.refreshHud();

// decay of health and fun over time
this.timerEventForStateDecay = this.time.addEvent({
    delay: 1000,
    repeat: -1, // it will repeat forever
    callback: () => {
        // update stats & refresh hud
        this.updateStats(this.decayRates);
        this.refreshHud();
    },
    callbackScope: this,
});

// follow pointer (mouse/finger) when dragging
this.input.on(Phaser.Input.Events.DRAG, (pointer, gameObject, dragX, dragY) => {
    // make sprite be located at the coordinates of the dragging
    gameObject.setPosition(dragX, dragY);
});
}

createUi() {
    // buttons
    this.appleBtn = this.add.sprite(72, 570, 'apple').setInteractive();
    this.appleBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.appleBtn); });
    this.appleBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 20, fun: 0 });

    this.candyBtn = this.add.sprite(144, 570, 'candy').setInteractive();
    this.candyBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.candyBtn); });
    this.candyBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: -10, fun: 10 });

    this.toyBtn = this.add.sprite(216, 570, 'toy').setInteractive();
    this.toyBtn.on(Phaser.Input.Events.POINTER_DOWN, () => { this.pickItem(this.toyBtn); });
    this.toyBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 0, fun: 15 });

    this.rotateBtn = this.add.sprite(288, 570, 'rotate').setInteractive();
    // the third argument allows us to pass what scope we want to be bound to the method provide
    this.rotateBtn.on(Phaser.Input.Events.POINTER_DOWN, this.rotatePet, this);
    this.rotateBtn.setData(CUSTOM_DATA_KEYS.STATS, { health: 0, fun: 10 });

    this.uiButtons = [this.appleBtn, this.candyBtn, this.toyBtn, this.rotateBtn];
}
```



```
this.isUiLocked = false;
this.selectedItem = null;
this.placedItem = undefined;
}

// pick item
pickItem(item) {
    // "this" scope is now referring to the GameScene instance

    // the ui can't be locked in order to select an item
    if (this.isUiLocked) {
        return;
    }

    // make sure the ui is ready
    this.refreshUi();

    // keep track of the selected item
    this.selectedItem = item;

    // change transparency
    item.setAlpha(0.5);

    console.log('we are picking ', item.texture.key);
    console.log(item.getData(CUSTOM_DATA_KEYS.STATS));
}

// rotate pet
rotatePet() {
    // "this" scope is referring to the GameScene instance

    // the ui can't be locked in order to select an item
    if (this.isUiLocked) {
        return;
    }

    // make sure the ui is ready
    this.refreshUi();

    // lock the ui
    this.isUiLocked = true;

    // dim the rotate icon
    this.rotateBtn.setAlpha(0.5);

    // rotation tween
    this.tweens.add({
        targets: this.pet,
        angle: 720,
        duration: 600,
        onComplete: () => {
            // increase fun
            const customStats = this.rotateBtn.getData(CUSTOM_DATA_KEYS.STATS);
            this.updateStats(customStats);
        }
    });
}
```



```
        // set UI to ready
        this.refreshUi();
    },
    callbackScope: this,
  });

  console.log('we are rotating the pet!');
}

// unlock ui
refreshUi() {
  // nothing is being selected
  this.selectedItem = null;

  // set all buttons to alpha 1 (no transparency)
  this.uiButtons.forEach((button) => {
    button.setAlpha(1);
  });

  // scene must be unlocked
  this.isUiLocked = false;

  // update hud
  this.refreshHud();
}

placeItem(pointer, localX, localY) {
  // scene must be unlocked
  if (this.isUiLocked) {
    return;
  }
  // check that an item was selected
  if (this.selectedItem === null) {
    return;
  }

  this.isUiLocked = true;

  // create a new item in the position the player clicked/tapped
  if (this.placedItem !== undefined) {
    this.placedItem.setPosition(localX, localY);
    this.placedItem.setTexture(this.selectedItem.texture.key);
    this.placedItem.setVisible(true);
  } else {
    this.placedItem = this.add.image(localX, localY, this.selectedItem.texture.key)
  }

  this.tweens.add({
    targets: this.pet,
    x: this.placedItem.x,
    y: this.placedItem.y,
    duration: 500,
    onComplete: () => {
      // hide our item game object
    }
  });
}
```



```
        this.placedItem.setVisible(false);
        // play spritesheet animation
        this.pet.play('funnyfaces');
        // increase stats
        const customStats = this.selectedItem.getData(CUSTOM_DATA_KEYS.STATS);
        this.updateStats(customStats);
    },
    callbackScope: this,
  });
}

// stat updater
updateStats(customStats) {
  // flag to see if it's game over
  let isGameOver = false;

  // update the pet stats
  for (const [key, value] of Object.entries(customStats)) {
    // console.log(key, value);
    if (this.stats[key] !== undefined) {
      this.stats[key] += value;

      // stats can't be less than zero
      if (this.stats[key] <= 0) {
        this.stats[key] = 0;
        isGameOver = true;
      }
    }
  }
}

// check to see if the game ended
if (isGameOver) {
  this.handleGameOver();
}

createHud() {
  this.healthText = this.add.text(20, 20, 'Health: ', {
    fill: '#000000',
    fontSize: '24px',
    fontFamily: 'Arial',
  });

  this.funText = this.add.text(170, 20, 'Fun: ', {
    fill: '#000000',
    fontSize: '24px',
    fontFamily: 'Arial',
  });
}

refreshHud() {
  this.healthText.setText(`Health: ${this.stats.health}`);
  this.funText.setText(`Fun: ${this.stats.fun}`);
}
```



```
handleGameOver() {
  // block ui
  this.isUiLocked = true;

  // cleanup timer event for stat decay
  this.timerEventForStateDecay.destroy();

  // change frame of the pet
  this.pet.setFrame(4);

  // keep the game on for a some time, the move on
  this.time.addEvent({
    delay: 2000,
    repeat: 0,
    callback: () => {
      this.scene.start('HomeScene');
    },
    callbackScope: this,
  });
}
```

home-scene.js

Found in folder: **/src/scenes**

This code defines a HomeScene class that extends Phaser.Scene. In the create method, it sets up the background image, a text label, and a rectangle border around the text. When the background is clicked, it starts the 'GameScene'.

```
export class HomeScene extends Phaser.Scene {
  constructor() {
    super({ key: 'HomeScene' });
  }

  create() {
    const { width, height } = this.scale;

    const bg = this.add.sprite(0, 0, 'backyard').setOrigin(0, 0).setInteractive();

    const text = this.add.text(width / 2, height / 2, '? VIRTUAL PET', {
      fill: 'ffffff',
      fontSize: '40px',
      fontFamily: 'Arial'
    }).setOrigin(0.5).setDepth(1);
    this.add.rectangle(text.x, text.y, text.width + 20, text.height + 20, 0x000000, 0.7);

    bg.once(Phaser.Input.Events.POINTER_DOWN, () => {
      this.scene.start('GameScene');
    }, this);
  }
}
```



loading-scene.js

Found in folder: **/src/scenes**

This code defines a LoadingScene class that extends Phaser.Scene. The preload() method loads game assets such as images and spritesheets, and displays a progress bar that fills as the assets load. The create() method creates an animation using the loaded spritesheet and starts the HomeScene.

```
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  preload() {
    // game width and height
    const { width, height } = this.scale;
    // show logo
    this.add.image(width / 2, 250, 'logo');

    const barW = 150;
    const barH = 30;
    // progress bar background
    this.add.rectangle(width / 2, height / 2, barW, barH, 0xf5f5f5, 1);
    // progress bar
    const progressBar = this.add.rectangle(width / 2 - barW/2, height / 2, 0, barH, 0x9ad98d, 1).setOrigin(0, 0.5);

    // load assets
    this.load.image('backyard', 'assets/images/backyard.png');
    this.load.image('apple', 'assets/images/apple.png');
    this.load.image('candy', 'assets/images/candy.png');
    this.load.image('rotate', 'assets/images/rotate.png');
    this.load.image('toy', 'assets/images/rubber_duck.png');

    // load spritesheet
    this.load.spritesheet('pet', 'assets/images/pet.png', {
      frameWidth: 97,
      frameHeight: 83,
      margin: 1,
      spacing: 1,
    });

    // TESTING - to see the progress bar in action!
    // for (let i = 0; i < 2000; i += 1) {
    //   this.load.image(`toy${i}`, 'assets/images/rubber_duck.png');
    // }

    // listen to the "progress" event
    this.load.on(Phaser.Loader.Events.PROGRESS, (value) => {
      progressBar.setSize(value * barW, barH);
    });
  }
}
```



```
create() {
    // create animation
    this.anims.create({
        key: 'funnyfaces',
        frames: this.anims.generateFrameNames('pet', { frames: [1, 2, 3] }),
        frameRate: 7,
        yoyo: true,
        repeat: 0, // to repeat forever: -1
    });

    this.scene.start('HomeScene');
}
}
```

index.html

Found in folder: /

This HTML file sets up the structure for a web page titled “?? Virtual Pet”. It includes two JavaScript files: “phaser.js” from the “src/lib” directory, and “main.js” from the “src” directory, with the latter being loaded as a module. The page’s background is set to black using CSS, and the default padding and margin are removed.

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <title>? Virtual Pet</title>
    <style>
        html, body {
            background: black;
            padding: 0px;
            margin: 0px;
        }
    </style>
</head>
<body>
    <script src="src/lib/phaser.js"></script>
    <script src="src/main.js" type="module"></script>
</body>
</html>
```